

Nota: O exame deve ser respondido em 3 folhas de prova separadas:

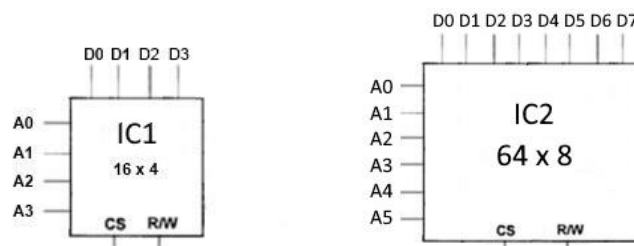
- 1 Folha para responder às três perguntas Teóricas
- 1 Folha para responder à 1ª pergunta da Prática
- 1 Folha para responder à 2ª pergunta da Prática

Duração Total Exame (T + P) : 2h:00m tolerância

13 de Junho de 2025

Parte Teórica

- 1** Considere os circuitos integrados de memória RAM da figura, onde $A_5, A_4, A_3, A_2, A_1, A_0$ representam linhas de endereço. Por seu lado, $D_7, D_6, D_5, D_4, D_3, D_2, D_1, D_0$ representam linhas de dados, R/W representa a linha de leitura/escrita e CS a linha de *Chip Selection*.



Faça um esboço, associando um número mínimo de circuitos integrados do tipo IC1, de forma a obter uma memória RAM com a capacidade e características do circuito IC2, identificando claramente as linhas de dados, de endereços, bem como as linhas R/W e CS da nova memória IC2.

(2 Val)

- 2** A tecnologia de *pipeline* é uma técnica amplamente utilizada em processadores para melhorar o desempenho da execução de instruções.
- Explique o que é *pipeline* no contexto de processadores.
 - Apresente um exemplo de como uma sequência de instruções em Assembly 8086 poderia beneficiar de uma execução com *pipeline*.
 - Identifique, e explique, dois tipos de problemas (limitações) que podem ocorrer em uma *pipeline* durante a execução de um programa Assembly. Refira ainda que soluções foram implementadas para contornar estas limitações.
- 3** Considere uma máquina com uma cache com mapeamento direto, blocos de 1 Byte e com 16 bits de *tag*. Esta máquina tem uma memória RAM com 1GB de capacidade.
- Qual a função da *tag*? Justifique. **(0,5 Val.)**
 - Qual a função dos *valid bits*? Justifique. **(0,5 Val.)**
 - Qual a capacidade total desta cache, contando com os bits da *tag* mais os *valid bits*? **(2 Val.)**

Parte Prática

1. De forma a ajudar à avaliação da turma de uma unidade curricular, foi desenvolvido um programa em Assembly 8086 que utiliza vetores (com os alunos que foram a exame e as suas notas) e, com base nos mesmos, realiza algumas estatísticas importantes. Infelizmente houve um problema no disco do computador onde estava o programa e todos os ficheiros, incluindo o do código fonte, ficaram corrompidos. Foi possível recuperar parte do código (apresentado abaixo), mas é necessário completá-lo de forma a responder às necessidades descritas nas alíneas do exercício.

```
.8086
.model small
.stack 2048
DADOS SEGMENT
    TURMA      10,9,19,11,5,20,22,28,24,13,3,23,8,16,4,7,1,12,0
    ALUNOS      24,4,7,12,1,13,0
    NOTAS       19,10,12,10,19,11
    TOTAL WORD
    PASSADOS
    REPROVADOS WORD
    MEDIA
    MELHORALUNO
    MELHORNOTA
DADOS ENDS

CODIGO SEGMENT PARA PUBLIC 'code'
    ASSUME     :CODIGO, DS:
main PROC
    MOV     , DADOS
    MOV DS, AX

    MOV AH, 4CH
    INT 21H
main ENDP
CODIGO ENDS
END main
```

Complete o programa recuperado para:

- Contabilizar o número de alunos da turma, o número de alunos que passaram à unidade curricular e que reprovaram. Estes valores devem ficar guardados nas variáveis TOTAL, PASSADOS e REPROVADOS, respetivamente.
- Calcular a média dos alunos que passaram (considere apenas a parte inteira da média). Este valor deve ficar guardado na variável MEDIA.
- Guardar a melhor nota e o id do melhor aluno nas variáveis MELHORNOTA e MELHORALUNO, respetivamente. No caso de existir mais do que um aluno com a nota mais alta, só deve ser tido em consideração o id do primeiro a ser identificado.

Considere que:

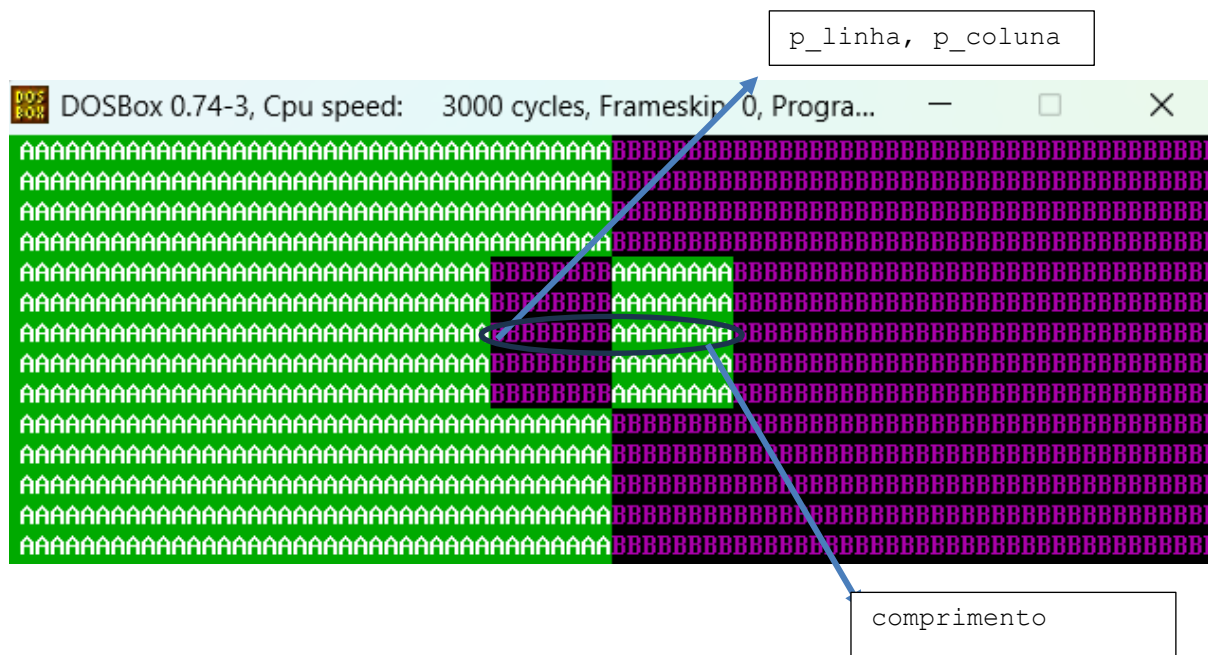
- O vetor TURMA contém o indentificador de cada aluno ($0 < \text{id} < 201$) da turma e pode ter no máximo 200 indivíduos. O vetor ALUNOS contém o id dos alunos que passaram à cadeira e o vetor NOTAS contém as respetivas notas. A nota de um determinado aluno no vetor ALUNOS está na mesma posição no vetor NOTAS (mesmo índice).
- Os vetores TURMA e ALUNOS não estão ordenados, nem têm um número fixo de elementos. O vetor ALUNOS, tal como o vetor TURMA, terminam com o valor 0 e o vetor notas contém apenas as notas de cada aluno que obteve aproveitamento à unidade curricular (valor natural > 9).

Atenção: deve copiar o código existente e acrescentar apenas o necessário para que o programa possa ser assemblado/linkado corretamente, cumprindo os objetivos expostos.

2. Considere um programa em Assembly 8086 que manipula diretamente a memória de vídeo para trocar metades de um retângulo de texto. O programa deverá ter o seguinte segmento de dados:

```
5  dseg      segment para public 'data'
6      p_linha      db 5          ; linha inicial
7      p_coluna     db 32         ; coluna inicial
8      comprimento  db 15         ; pode ser ímpar – será ajustado
9      largura      db 5          ; número de linhas
10     buffer       dw 40 dup(?)  ; espaço temporário
11  dseg      ends
```

Caso a metade vertical do lado esquerdo do monitor esteja previamente preenchida com “A” e a direita com “B”, a posição inicial do retângulo (canto superior esquerdo) seja dada pelo par (p_linha, p_coluna) e os seus lados pelas variáveis comprimento e largura, o efeito do programa a desenvolver é apresentado abaixo: a parte direita do quadrado (na vertical) é copiada para o lado esquerdo e vice-versa.



- Escreva o procedimento `PosicaoInicial` que calcule o endereço inicial no segmento de vídeo correspondente à posição no ecrã (p_linha, p_coluna). O valor da posição deve ser armazenado em SI.
- Escreva o procedimento `CorrigirComprimento` que verifica se o valor da variável `comprimento` é ímpar. Caso seja, incremente-a para que fique par.
- O procedimento `CopiaLinha` permite para cada linha, na área definida como `comprimento`, copiar a metade esquerda da linha para o lado direito e vice-versa.

Inicial: AAAAAAAABBBBBBBB
Final: BBBBBBBBBAAAAAAA

Para isso, recebe a posição inicial de uma linha de caracteres no ecrã de vídeo e troca as metades da linha, pela seguinte ordem:

- Guarda a metade esquerda em `buffer`.
- Copia a metade direita para o lado esquerdo.
- Restaura a metade esquerda do `buffer` no lado direito.

Sabendo que:

- SI aponta para o início da linha no ecrã;
- CX contém o número de colunas a copiar (`comprimento / 2`);
- Buffer é um vetor auxiliar de words.

Complete os blocos de código em falta abaixo:

- Guarda cada par (caracter + atributo) da metade esquerda da linha no `buffer`.

CopiaLinha `proc`

`; -----`

`; Copiar metade esquerda → buffer`

```
    xor di, di    ; índice no buffer
    push cx       ; número de colunas a copiar (comprimento / 2)
```

`copy_left to buffer:`

```
    loop copy_left_to_buffer
```

`; -----`

- Escreva o código que calcula a posição onde começa a metade direita da linha no ecrã e guarde-o em DI.
- Complete o ciclo que copia caracteres da metade direita da linha para o lado esquerdo (utilizando os valores de SI e DI das alíneas anteriores).

```
    push cx
```

`copy_right_to_left:`

```
    loop copy_right_to_left
    pop cx
```

- No `main` do programa escreva as instruções em falta (relativas a CX e SI) para que todas as linhas do retângulo (`largura`) possam ser executadas.

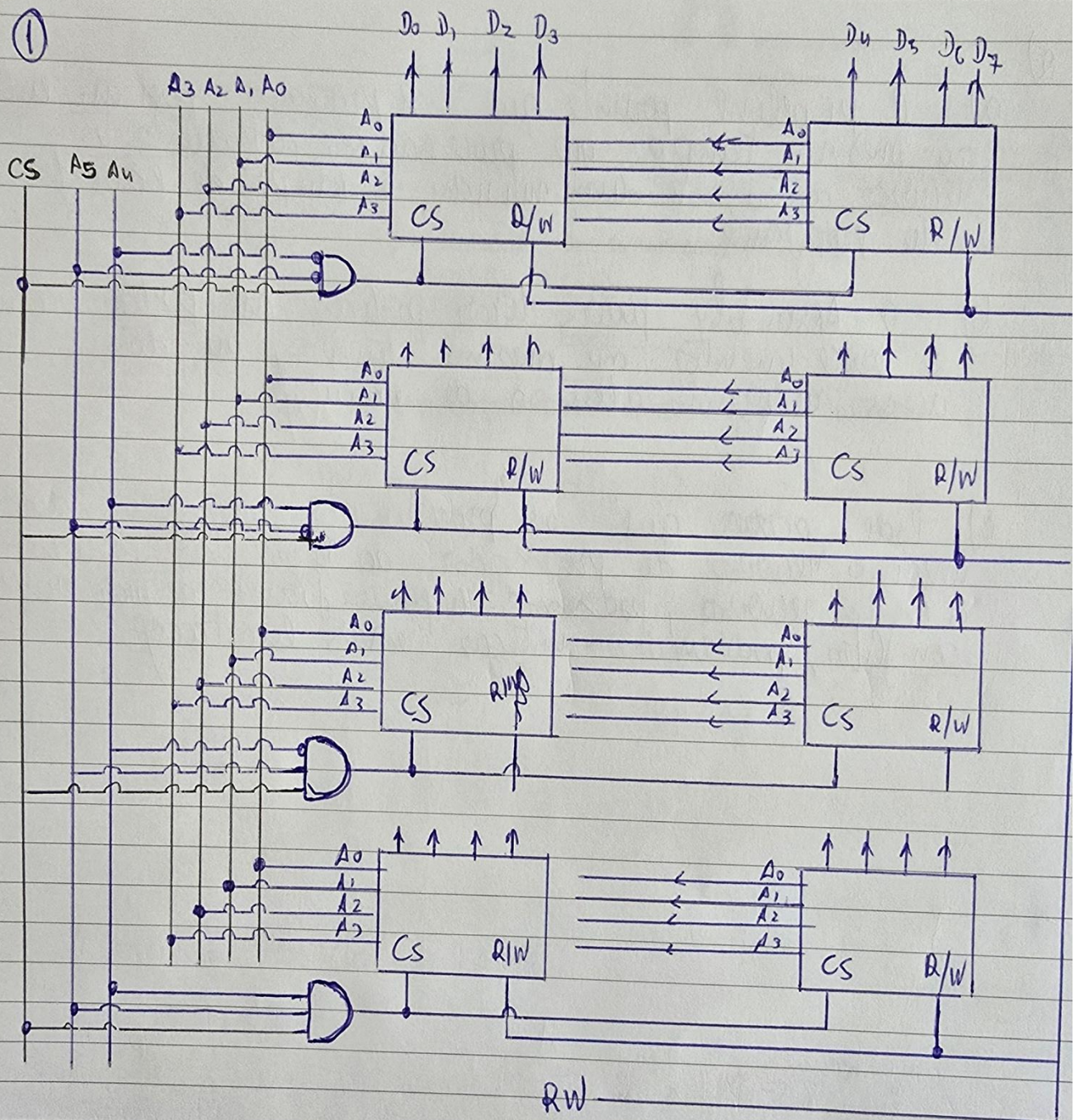
```

15  main    proc
16          mov     ax, dseg
17          mov     ds, ax
18          mov     ax, 0B800h
19          mov     es, ax
20
21          call PosicaoInicial
22          call CorrigirComprimento
23          ; -----
24          ; ciclo principal por linha
25          ; -----
26          xor     ch, ch
27          mov     cl, largura
28          linha_loop:
29              |
30              |
31              |
32              |
33              |
34              |
35              |
36              |
37              |
38              |
39              |
40          loop linha_loop
41
42  fim:     mov     ah, 4ch
43          int     21h
44  main    endp

```

BOA SORTE! ☺

①

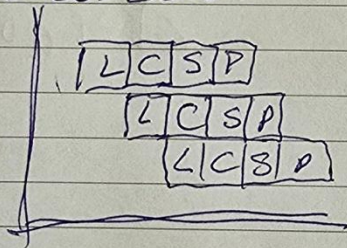


2) No contexto de processadores

pipeline é a técnica usada que permite transformar os instruções em etapas que podem ser executadas em paralelo.

Usando a analogia da lavandaria, se o processo de lavagem inclui etapas de

(4) lavagem (cleaning), (5) Secagem (drying) e (6) dobragem (folding) então poderá ser executadas em paralelo pelo processador



~~A) Se existirem etapas de espera de tempo~~

b) Se existem etapas num processo diferenciáveis elas poderão ser executadas em paralelo por exemplo num ciclo que efectua a mesma ação sobre os elementos iterados.

c) Se as etapas não são perfeitamente possíveis de individualizar e têm de se fazer os processos de forma independente uma forma é fazer dois processos em paralelo em threads diferentes

3)

a) A função da tag é identificar os bits mais significativos do endereço de memória RAM, ao qual corresponde uma determinada célula de cache.

b) Cada célula de cache tem um valid bit, que permite identificar se o seu valor é válido, ou seja, se já foi utilizada e se está de acordo com o seu valor correspondente na memória RAM.

c)

16 bits de tag

1 GB de memória RAM — $2^0 \times 2^{30}$ — 30 bits de endereçamento RAM

Bits RAM = bits de tag + bits da cache $\Rightarrow 30 = 16 + \text{bits da cache}$

$\Rightarrow$ bits da cache = 14

$$2^{14} = 2^4 \times 2^{10} = 16 \text{ KBytes}$$

Cap. total de cache = $n^\circ \text{ Valid Bits} \times \text{Cap. da cache em bits} +$
 $+ n^\circ \text{ Bits tag} \times \text{cap. da cache em bits}$
 $+ \text{Cap. da cache}$

$$\text{Cap. total de cache} = 1 \times 16 \text{ Kb} + 16 \times 16 \text{ Kb} + 16 \text{ KB}$$

$$= \frac{16}{8} \text{ KB} + \frac{16}{8} \times 16 \text{ KB} + 16 \text{ KB}$$

$$= 2 \text{ KB} + 32 \text{ KB} + 16 \text{ KB} = 50 \text{ KBytes}$$

R: A capacidade total deste cache é de 50 KBytes